

Apple Technical Round – Last-Minute Revision

Keep these as **flash cards**. For most questions, answer the definition first, then add **one example/trade-off**.

1. Core CS – OS

Topic	1-2 line revision
Process vs Thread	Process has separate address space; threads share process memory. Threads are cheaper but require synchronization.
Context Switch	CPU saves one execution context and loads another; too many switches hurt performance.
Race Condition	Output depends on timing of concurrent operations; solve with mutex/atomic/appropriate synchronization.
Deadlock	Threads wait forever for each other's resources; prevent with lock ordering, <code>scoped_lock</code> , timeouts.
Mutex	Provides mutual exclusion: only one thread enters the critical section at a time.
Semaphore	Controls access using a count of permits; useful for limiting concurrent resources.
Condition Variable	Lets a thread sleep until a condition becomes true; common in producer-consumer/thread pools.
Virtual Memory	Gives processes isolated virtual address spaces; OS maps virtual pages to physical memory.
Paging	Memory is divided into fixed-size pages/frames; page faults occur when required data isn't in RAM.
Stack vs Heap	Stack is automatic/scoped and usually fast; heap is dynamically managed and has allocation overhead.
Mutex vs Atomic	Mutex protects compound/shared invariants; atomic is ideal for simple independent atomic operations.
Starvation	A thread waits indefinitely for resources/CPU; fairness and scheduling policies can reduce it.

Key mapping

Concurrency

- └─ Mutex → protect critical section
- └─ Atomic → simple shared variable
- └─ CV → wait efficiently
- └─ Semaphore → limit resource count

2. Core CS – DBMS

Topic	Revision
Primary Key	Uniquely identifies a row; should be stable and non-null.
Foreign Key	Enforces relationship between tables and referential integrity.
Index	Speeds reads by maintaining a searchable structure; costs storage and slows writes.
Composite Index	Index on multiple columns; column order matters for query usability.
Normalization	Reduces redundancy and update anomalies; too much normalization can increase joins.
ACID	Atomicity, Consistency, Isolation, Durability – core transactional guarantees.
Transaction	Group of operations treated as one logical unit.
Isolation	Controls how concurrent transactions observe each other's changes.
MVCC	PostgreSQL keeps row versions so readers/writers can often proceed concurrently.
Deadlock	Transactions wait on each other's locks; DB may abort one transaction.
EXPLAIN ANALYZE	Shows the actual query execution plan and timings; use it to diagnose slow SQL.
Connection Pool	Reuses DB connections and bounds DB concurrency.

Key mapping

Slow query

↓

EXPLAIN ANALYZE

↓
Scan / Join / Sort / Index?
↓
Optimize query or index
↓
Measure again

3. Core CS – OOP

Topic	Revision
Encapsulation	Bundle data + behavior and control access to internal state.
Abstraction	Expose what an object does while hiding implementation details.
Inheritance	Derive a class from another; use carefully because composition is often more flexible.
Polymorphism	Same interface, different implementations; virtual functions provide runtime polymorphism.
Virtual Function	Enables dynamic dispatch based on actual object type.
Constructor	Initializes object; destructor releases resources.
RAII	Resource lifetime is tied to object lifetime; fundamental for safe C++ resource management.
Composition vs Inheritance	Composition = “has-a”; inheritance = “is-a”. Prefer composition when appropriate.

4. Core CS – C++

Topic	Revision
Pointer	Stores an address; can be null and requires careful lifetime management.
Reference	Alias to an existing object; normally must refer to a valid object.
<code>const</code>	Prevents modification through that interface and communicates intent.
<code>unique_ptr</code>	Single ownership; preferred smart pointer when ownership isn't shared.
<code>shared_ptr</code>	Shared ownership using reference counting; don't use unless ownership is genuinely shared.

Topic	Revision
Move Semantics	Transfers resources instead of copying them; useful for performance.
<code>std::move</code>	Casts to an rvalue; actual move happens via move constructor/assignment.
Copy vs Move	Copy duplicates resources; move transfers them when possible.
Undefined Behavior	Standard imposes no guarantees; examples include out-of-bounds access and data races.
STL	Know <code>vector</code> , <code>deque</code> , <code>list</code> , <code>set</code> , <code>unordered_set</code> , <code>map</code> , <code>unordered_map</code> , <code>priority_queue</code> .
<code>vector</code>	Contiguous memory, fast random access, amortized $O(1)$ <code>push_back</code> .
<code>unordered_map</code>	Average $O(1)$ lookup, worst-case $O(n)$; hash-table based.
<code>map</code>	Balanced-tree structure; $O(\log n)$ search/insert/erase.

High-value C++ mapping

Ownership → `unique_ptr` / `shared_ptr`
 Non-owning → `reference` / `raw pointer`
 Resource life → `RAII`
 Performance → `move semantics`
 Shared state → `mutex` / `atomic`

5. Core CS – Computer Networks

Topic	Revision
TCP	Reliable, ordered byte stream; connection-oriented.
UDP	Connectionless datagrams; no delivery/order guarantee, lower overhead.
HTTP	Application-layer protocol generally built over TCP/TLS (HTTP/3 uses QUIC).
HTTPS	HTTP secured with TLS.
TCP 3-Way Handshake	<code>SYN</code> → <code>SYN-ACK</code> → <code>ACK</code> .
DNS	Maps domain names to IP addresses.

Topic	Revision
Latency vs Throughput	Latency = time per operation; throughput = operations/data per unit time.
Load Balancer	Distributes requests across service instances.
REST	Resource-oriented API style using standard HTTP methods/status codes.

HTTP methods

```

GET      → read
POST     → create/action
PUT      → replace
PATCH   → partial update
DELETE   → delete

```

6. Core CS – Algorithms

Topic	Revision
Binary Search	$O(\log n)$, requires a monotonic/searchable condition.
BFS	Level-order traversal; shortest path in an unweighted graph.
DFS	Deep traversal; useful for connectivity, cycle detection, components.
Dijkstra	Shortest path with non-negative edge weights.
Bellman-Ford	Handles negative edges and can detect negative cycles.
Topological Sort	Ordering of DAG vertices such that every edge goes forward.
Heap	Supports min/max extraction efficiently; typical push/pop $O(\log n)$.
Hashing	Average $O(1)$ lookup; collision handling is essential.
Two Pointers	Often $O(n)$ for sorted arrays/string windows.
Sliding Window	Maintains a moving range; useful for subarray/substring constraints.
Greedy	Make locally optimal choices; must justify correctness.
DP	Define state + transition + base case; avoid repeated subproblems.
Union-Find	Near $O(1)$ amortized connectivity queries with path compression + union by rank/size.

DSA mapping

Shortest path:

unweighted → BFS

positive weights → Dijkstra

negative edges → Bellman-Ford

Connectivity:

static → DFS/BFS

dynamic unions → DSU

Subarray/string:

sorted/paired → two pointers

contiguous constraint → sliding window

repeated subproblems → DP

7. Your HPE Project – Highest Priority

Your resume says you worked on a **C++ authorization service, Kafka event processing, custom thread pool, PostgreSQL connection pool, and SQL optimization.**

Memorize this architecture

Payment Producer



Kafka



Kafka Consumer



Bounded Work Queue



C++ Thread Pool



Validation / Authorization



PostgreSQL Connection Pool



PostgreSQL

Question	One-line answer
Why Kafka?	Decouples producers/consumers, buffers traffic spikes, enables replay/scaling.
Why thread pool?	Avoids thread creation per task and provides controlled concurrency.
Why bounded queue?	Prevents unbounded memory growth and provides backpressure.
Why DB pool?	Reuses connections and limits DB concurrency.
Duplicate Kafka message?	Use idempotent processing + unique transaction ID/database constraint.
Consumer crash before commit?	Message can be redelivered; design processing to be idempotent.
Poison message?	Retry transient errors; dead-letter permanent failures after bounded retries.
Partition key?	Choose the smallest business entity that requires ordering, e.g. account ID.
DB bottleneck?	Don't add threads blindly; optimize SQL/indexes and apply backpressure.
Performance metrics?	p50/p95/p99 latency, throughput, DB latency, consumer lag, queue depth.

Golden mapping

Kafka → durability + decoupling + scaling

Thread Pool → controlled CPU/application concurrency

Connection Pool → controlled DB concurrency

Idempotency → duplicate protection

Backpressure → overload protection

Indexes → read optimization

8. Invest-Tracker

Your project uses **React, Node/Express, PostgreSQL/Supabase, external APIs, JWT and Row Level Security**.

React

↓

Express REST API

↓
Auth Middleware
↓
PostgreSQL / Supabase
↓
External APIs

Topic	Revision
JWT	Signed token proving claims haven't been modified; always verify signature + expiry.
Authentication	"Who are you?"
Authorization	"What are you allowed to do?"
HTTP-only cookie	JavaScript cannot directly read it; helps reduce token theft through XSS.
RLS	Database itself restricts which rows a user can access.
IDOR	Never trust object IDs from client; enforce ownership server/database-side.
External API failure	Timeout, retry transient errors, backoff, cache, graceful degradation.
API rate limit	Cache/reuse data and control request frequency to avoid provider throttling.
CORS	Browser security mechanism controlling cross-origin requests.

Security mapping

Authentication → JWT
Authorization → middleware / role checks
Data isolation → PostgreSQL RLS
Cookie safety → HttpOnly + Secure + SameSite
Password → bcrypt hash

9. Quick-Pick – E-Commerce

Your project covers **MongoDB, JWT, bcrypt, RBAC, Cloudinary and PayPal.**

Topic	Revision
bcrypt	Password hashing; passwords should generally be hashed, not reversibly encrypted.

Topic	Revision
RBAC	Permissions based on roles such as user/admin.
HTTP-only JWT cookie	Authentication credential inaccessible to normal JS APIs.
Payment duplicate request	Use idempotency key to make retry safe.
Payment succeeds, DB fails	Treat as distributed consistency problem; use durable states + retry/reconciliation.
MongoDB	Document-oriented; useful when data/access patterns fit document modeling.
Cloudinary	External media storage/CDN service; don't overload application server with image storage.
Order state machine	CREATED → PAYMENT_PENDING → CONFIRMED → COMPLETED/FAILED.

10. Node.js / React

Topic	Revision
Node.js event loop	Efficient for I/O concurrency; CPU-heavy JS can block the event loop.
Promise	Represents eventual completion/failure of an async operation.
Middleware	Function in request pipeline; useful for auth, logging, validation, etc.
React state	State changes trigger rendering/reconciliation.
React key	Stable identity for list elements; avoids incorrect reconciliation.
Re-render optimization	Profile first; then use state localization, memoization, virtualization where justified.
CORS	Browser-enforced cross-origin access policy, not an authentication mechanism.

11. Resume Rapid-Fire

Know these **without thinking**:

Resume item	Expected follow-up
-------------	--------------------

Resume item	Expected follow-up
C++	RAII? move? smart pointers? virtual functions? concurrency?
Kafka	partition? offset? consumer group? rebalance? retries?
Thread Pool	queue? CV? shutdown? starvation? deadlock?
PostgreSQL	indexes? transactions? isolation? MVCC? EXPLAIN ANALYZE ?
SQL optimization	What was slow? What changed? How did you measure it?
JWT	signing? expiry? cookie? XSS? CSRF?
RLS	How is user identity enforced at DB layer?
MongoDB	Why Mongo instead of SQL?
PayPal	duplicate callback? timeout? payment/DB consistency?
React	state? props? keys? re-render?
Node.js	event loop? blocking CPU? middleware?
REST	PUT vs PATCH? status codes? idempotency?
Vercel	deployment architecture? CORS? environment variables?

12. HTTP Status Codes – Memorize

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthorized → authentication missing/invalid

403 Forbidden → authenticated but not allowed

404 Not Found

409 Conflict → state/uniqueness conflict

429 Too Many Requests

500 Internal Server Error

502 Bad Gateway

503 Service Unavailable

Classic trap: 401 ≠ 403 .

13. Most Important “Mapping” Sheet

Concurrency

Need exclusive access	→ Mutex
Need atomic counter	→ Atomic
Need wait for condition	→ Condition Variable
Need N permits	→ Semaphore
Many tasks	→ Thread Pool
Too much incoming work	→ Backpressure

Kafka

Topic	→ stream/category
Partition	→ ordered unit + scalability unit
Offset	→ position in partition
Consumer	→ reads messages
Group	→ load-sharing consumers
Lag	→ falling behind
Redelivery	→ idempotency required

Database

Slow reads	→ Index / Query optimization
Too many DB conns	→ Connection Pool
Concurrent data	→ Transaction / Isolation
Row contention	→ Locking
User isolation	→ RLS
Duplicate record	→ Unique constraint

Security

Password	→ bcrypt hash
Authentication	→ JWT/session
Authorization	→ RBAC/policies
Data isolation	→ RLS

XSS → output/input defenses + HttpOnly reduces cookie theft
CSRF → SameSite / CSRF token where needed

Performance

Measure → Identify bottleneck → Optimize → Benchmark again

Latency → p50 / p95 / p99
Throughput → requests/events per second
Kafka health → consumer lag
Queue pressure → queue depth
DB health → query latency / connections

14. Final 30-Second Project Pitch

For **HPE**, keep this mental answer ready:

“I worked on a C++ authorization service for a payment gateway. We used Kafka for event-driven processing, a bounded thread pool for controlled concurrency, and a PostgreSQL connection pool to reduce connection overhead. I focused on correctness, idempotent processing, database efficiency, and measuring latency/throughput rather than simply increasing concurrency.”

For **Invest-Tracker**:

“I built a React/Node investment tracker with REST APIs, external market-data integrations, JWT authentication, and PostgreSQL Row Level Security for per-user isolation.”

For **Quick-Pick**:

“I built an e-commerce platform using React, Node/Express and MongoDB, with JWT/RBAC authentication, bcrypt password hashing, Cloudinary media storage, and PayPal checkout integration.”

Last-minute priority

If you have **only 30 minutes**, revise in this order:

Kafka → C++ concurrency → PostgreSQL → OS concurrency → JWT/RLS/security → DB transactions/indexes → REST/HTTP → Node.js → React → DSA complexities.

And remember this interview pattern:

“What did you build?” → “Why?” → “How internally?” → “What can fail?” → “How do you scale it?” → “How did you measure it?”

That six-step chain is the best way to survive deep project follow-ups.